



Side-Channel Analysis of Ascon-128 Hardware on ChipWhisperer CW305

A comparative study direction for HLS-generated hardware and
pure RTL hardware

Submitted by

Rupesh Hemant Bhusare

2203106, CSE

Topic Area: Hardware Security and Side-Channel Attacks

Acknowledgment

I would like to thank my project guide, faculty members, and lab staff for their support in hardware access, technical guidance, and project review. I also acknowledge the Chip-Whisperer ecosystem, the Ascon reference implementation, and the Xilinx Vivado/Vitis HLS toolchain used in this work.

Abstract

This Bachelor of Technology Project studies side-channel leakage from hardware implementations of the Ascon-128 authenticated encryption algorithm on the ChipWhisperer CW305 Artix-7 FPGA target. The project builds on earlier work completed on the ChipWhisperer Nano platform, where software-based attacks such as Simple Power Analysis (SPA), Differential Power Analysis (DPA), and Correlation Power Analysis (CPA) were applied to AES running on an STM32F0 microcontroller using the Hamming Weight leakage model. The current phase moves from software leakage to hardware leakage.

The present work selects Ascon-128 as the target because it is a standard lightweight authenticated encryption algorithm and is suitable for FPGA-based implementation. The hardware design path uses the `crypto_aead/ascon128v12/ref` reference C code and converts it into hardware through Vitis HLS 2024.2. A merged top-level function named `ascon_core` is used so that encryption and decryption both exist inside one hardware module with a mode control input. A Tcl automation script is used to explore multiple HLS solutions such as baseline, loop unrolling, pipelining, and dataflow.

The attack model planned for the FPGA phase is Correlation Power Analysis with a Hamming Distance leakage model. This choice is motivated by the fact that FPGA hardware often leaks through transitions between register values rather than only through absolute stored values. However, this report does not claim completed physical trace-capture results on CW305 or completed pure RTL comparison results. Instead, it presents the completed background, the completed HLS design stage, and the methodology for the next stage of physical hardware evaluation.

Contents

1	Introduction	6
2	Background and Previous Work	8
3	Theoretical Foundation	12
3.1	Side-Channel Leakage	12
3.2	Hamming Weight Model	12
3.3	Hamming Distance Model	12
3.4	Correlation Power Analysis	13
3.5	Why Hamming Distance for This Work	13
4	Problem Statement and Objectives	15
5	Platform and Design Flow	17
5.1	Available Boards	17
5.2	Local Project Files Used as Context	18
5.3	Design Flow	18
6	Ascon-128 Hardware Design	20
6.1	Why Ascon-128	20
6.2	Reference Code Choice	20
6.3	Unified Top Function	20
6.4	HLS Interface Pragmas	21
6.5	Operational Stages	21
7	Side-Channel Attack Methodology	24

7.1	Attack Choice	24
7.2	Leakage Model	24
7.3	Target Intermediate Selection	24
7.4	General CPA Flow	25
8	Experimental Setup	26
8.1	HLS Design-Space Exploration	26
8.2	Planned CW305 Hardware Test Flow	27
8.3	Planned Capture Conditions	28
9	Results and Analysis	29
9.1	Functional Validation	29
9.2	Synthesis Results	29
9.3	Future Hardware Measurement Stage	30
10	Comparison of HLS Hardware and RTL Hardware	31
10.1	Fair Comparison Rules	31
11	Challenges, Limits, and Validity	32
12	Conclusion and Future Work	33

Abbreviations

AEAD	Authenticated Encryption with Associated Data
AES	Advanced Encryption Standard
CPA	Correlation Power Analysis
CW305	ChipWhisperer CW305 Artix FPGA Target
CWNano	ChipWhisperer Nano
DPA	Differential Power Analysis
FPGA	Field Programmable Gate Array
HD	Hamming Distance
HLS	High-Level Synthesis
HW	Hamming Weight
LUT	Look-Up Table
RTL	Register Transfer Level
SCA	Side-Channel Attack
SPA	Simple Power Analysis

Chapter 1

Introduction

Side-channel attacks are attacks that do not break a cipher by solving only its mathematics. Instead, they study information leaked by the physical implementation of the cipher. Power consumption, timing, electromagnetic radiation, and cache activity are common leakage sources. In embedded and hardware security, power analysis is one of the most practical attack methods because it can often be performed using low-cost lab equipment.

In the previous phase of this BTP, attacks were performed on software running on a microcontroller. That phase established the basic ideas of leakage, trigger-based trace capture, and key recovery. In the current phase, the target is no longer a CPU running cryptographic code instruction by instruction. The target is hardware created on an FPGA. This is an important step because many real systems use hardware accelerators to improve speed and efficiency. Such accelerators may still leak secret-dependent information.

This project uses the ChipWhisperer CW305 Artix-7 FPGA board as the hardware target. The selected cryptographic algorithm is Ascon-128. The selected implementation path is High-Level Synthesis, where reference C code is converted into RTL hardware by Vitis HLS. The selected future attack path is Correlation Power Analysis using the Hamming Distance model. The longer-term goal is to compare the leakage behavior of HLS-generated hardware and pure RTL hardware after both are physically evaluated.

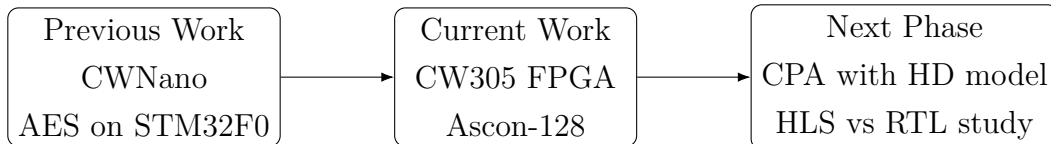


Figure 1.1: Project progression from software attacks to hardware design and future FPGA leakage study

Table 1.1: Previous phase and current phase of the BTP

Parameter	Previous Phase	Current Phase
Target board	ChipWhisperer Nano	ChipWhisperer CW305 Artix-7 FPGA
Target type	STM32F0 microcontroller	FPGA hardware core
Algorithm	AES	Ascon-128
Attack type	SPA, DPA, CPA	HLS design and planned CPA
Leakage model	Hamming Weight	Hamming Distance planned for FPGA
Implementation style	Firmware C on CPU	HLS-generated hardware, later to be compared with RTL

Chapter 2

Background and Previous Work

The first phase of the BTP was completed on the ChipWhisperer Nano platform. In that phase, the focus was on learning how software implementations leak through power traces. Different experiments were performed on an STM32F0 target, including simple instruction-level leakage, key-dependent leakage through the Hamming Weight model, and full key-recovery style attacks on AES firmware.

Simple Power Analysis was useful to show that even without advanced statistics, the shape of a trace can reveal useful information about control flow. For example, a password check or a multiplication-heavy code path may show a visible pattern difference in time or power. This helped build intuition about why secure implementations must avoid data-dependent behavior.

The next stage used Differential Power Analysis and later Correlation Power Analysis. The main target was AES firmware on the CWNano board. In that setting, random plaintexts were sent to the target, power traces were captured, and the first-round intermediate values were used to build leakage hypotheses. The Hamming Weight of the S-box output was treated as the expected leakage. Correlation was then computed between the predicted leakage and each time sample of the measured traces.

The CPA study on the Nano was more detailed and more reliable than the earlier DPA stage. In DPA, traces are usually divided into groups based on a one-bit prediction, and then average traces are subtracted. This can reveal leakage, but it is sensitive to noise and can produce weak or false peaks when the number of traces is limited. CPA improved the result because it used a fuller leakage model. Instead of checking one bit only, the attack predicted the full Hamming Weight of the intermediate value for each possible key guess. This gave a prediction that matched the physical behavior of the microcontroller more closely.

For the AES experiment, the first-round S-box output was used as the intermediate value because it depends on both the known plaintext and the secret key byte. For a plaintext

byte P_i and a guessed key byte k_g , the predicted intermediate value was:

$$v_i = S(P_i \oplus k_g)$$

where $S(\cdot)$ is the AES S-box. The expected leakage was then modeled as:

$$\ell_i = HW(v_i)$$

where $HW(\cdot)$ gives the number of one bits in the intermediate value. This follows the Hamming Weight assumption used for the Nano firmware target.

For each key guess, the Pearson Correlation coefficient was computed between the predicted leakage vector and the measured power values at each time sample. The formula used in the earlier CPA work was:

$$\rho_{X,Y} = \frac{\text{cov}(X,Y)}{\sigma_X \sigma_Y} \quad (2.1)$$

where X is the vector of predicted Hamming Weight values and Y is the vector of measured power samples at a fixed time index across all traces. A strong peak for one key guess indicated that the measured power changed in the same statistical pattern as the predicted intermediate values.

The CPA experiment on the Nano used 200 traces. This was an important practical choice because the ChipWhisperer Nano has a higher noise floor than larger capture platforms. With too few traces, random variation can hide the data-dependent leakage. Correlation-based analysis reduces uncorrelated noise because it looks for a consistent relationship across many traces rather than only comparing two grouped averages. In this earlier experiment, the use of 200 traces gave a clearer statistical result and made CPA more stable than DPA. It also reduced the chance of ghost peaks and helped identify the correct key hypothesis more cleanly.

This completed Nano CPA work is directly connected to the current BTP phase. It established the full attack workflow of input control, trace capture, leakage modeling, and statistical key testing. The key change in the present FPGA phase is not the attack philosophy, but the leakage model: the current hardware target is expected to follow Hamming Distance more naturally than Hamming Weight.

Table 2.1: Summary of previous semester work

Experiment	Platform	Main learning outcome
Instruction-level SPA	CWNano STM32F0	Different code patterns create different power shapes
Password/timing pattern study	CWNano STM32F0	Control-flow leakage can reveal secret-dependent behavior
AES DPA	CWNano STM32F0	Large trace sets can recover key information
AES CPA	CWNano STM32F0	Correlation with Hamming Weight gives stronger attack quality

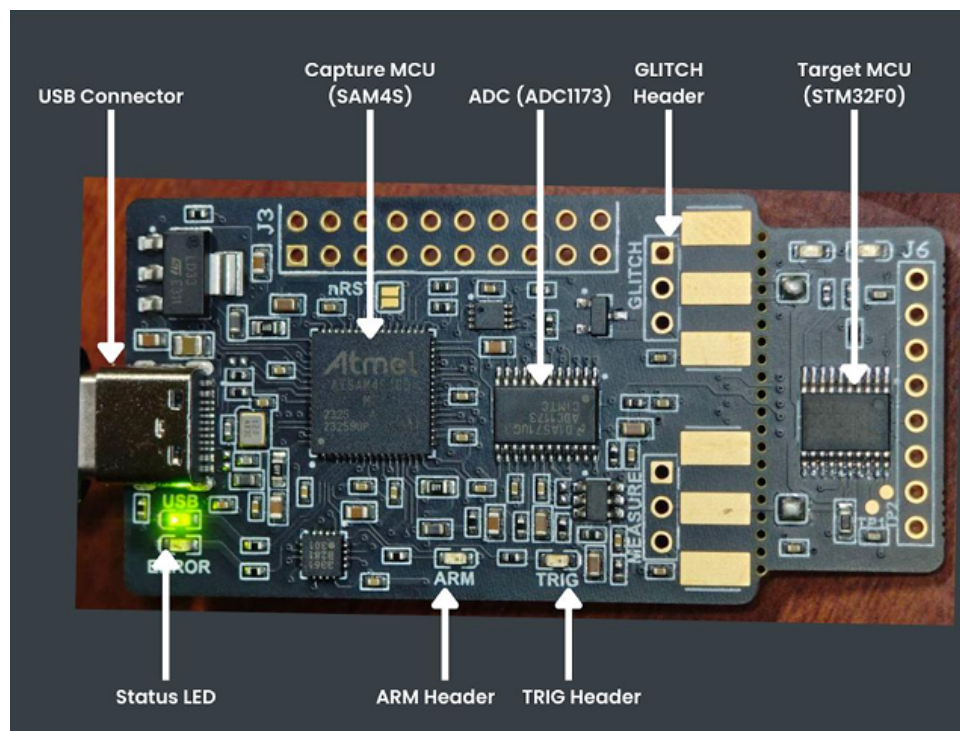


Figure 2.1: ChipWhisperer Nano board used in the previous semester for firmware-side side-channel experiments on the STM32F0 target.

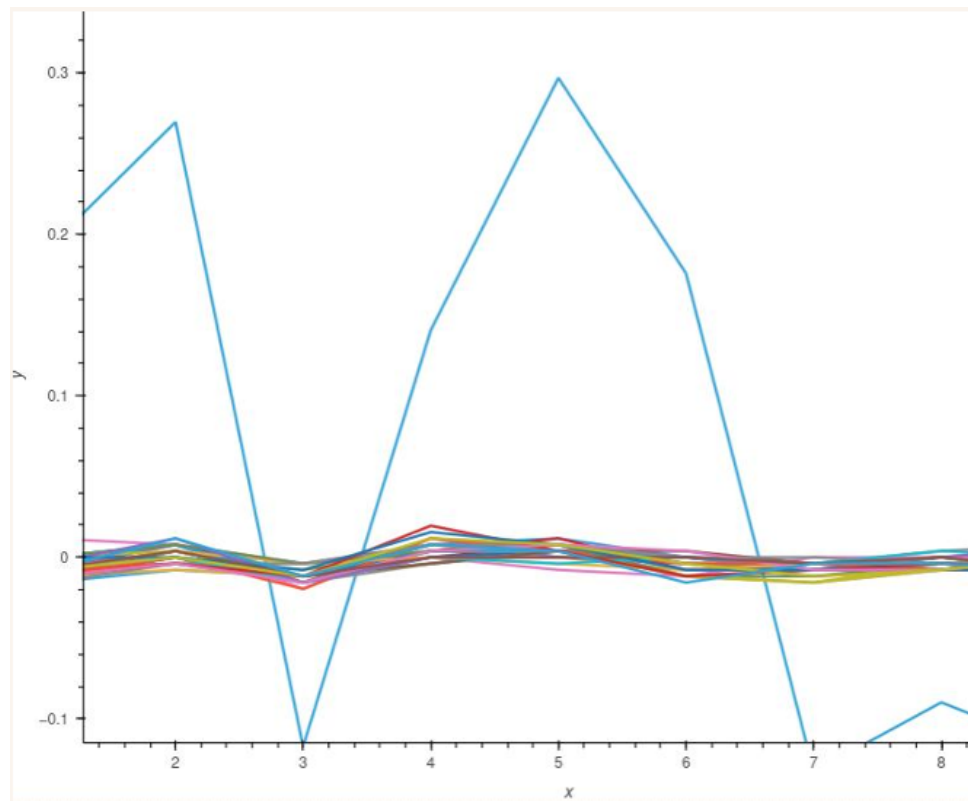


Figure 2.2: SPA power traces showing the difference between correct and wrong character cases in the earlier Nano-based firmware experiments.

Chapter 3

Theoretical Foundation

3.1 Side-Channel Leakage

An implementation leaks information when its physical behavior depends on secret data. In digital hardware, this often happens because bit transitions cause switching activity in registers, routing, and combinational logic. The more switching activity there is, the larger the dynamic power component can become. Because cryptographic operations process secret values, this switching can become correlated with those values.

3.2 Hamming Weight Model

The Hamming Weight of a binary word is the number of bits that are equal to one. The model assumes that power is related to the number of one bits stored in a register. This model is often useful in microcontroller attacks because data values are loaded and stored in a regular way.

3.3 Hamming Distance Model

The Hamming Distance between two binary words is the number of bit positions that change when one value moves to another. The model assumes that power is related to the number of switching bits during transitions. This is often a better model for FPGA and hardware logic because many nodes switch between consecutive internal states.

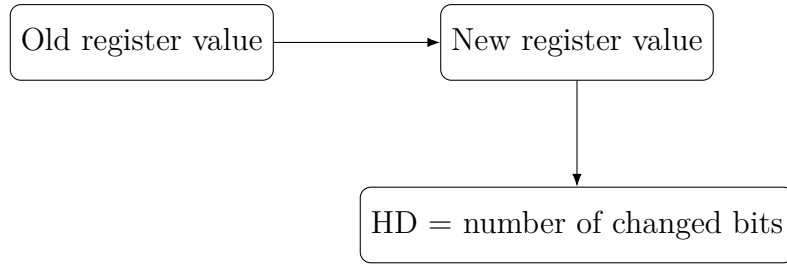


Figure 3.1: Basic idea of Hamming Distance leakage

3.4 Correlation Power Analysis

Correlation Power Analysis compares measured traces with predicted leakage values. For each key guess or internal-state guess, the attacker computes a hypothetical leakage vector. The Pearson correlation coefficient is then computed between the predicted vector and the measured power at each time point.

$$\rho_{X,Y} = \frac{\text{cov}(X,Y)}{\sigma_X \sigma_Y} \quad (3.1)$$

If the model is correct and the intermediate value is correctly chosen, the right guess produces a visible correlation peak at the time where the target operation occurs.

3.5 Why Hamming Distance for This Work

In the current project, the target is a hardware design on an FPGA. Since FPGA activity is strongly affected by signal transitions and register updates, the Hamming Distance model is more suitable than a pure Hamming Weight model for the main attack stage. The previous semester used Hamming Weight successfully for firmware AES, but the new hardware setting requires a model that follows transitions more directly.

Table 3.1: Comparison of Hamming Weight and Hamming Distance

Property	Hamming Weight	Hamming Distance
Meaning	Number of one bits in a value	Number of changed bits between two values
Best suited for	Stored-value leakage	Transition-based leakage
Typical target	Firmware registers and memory effects	FPGA registers, buses, switching logic
Use in this BTP	Previous AES firmware work	Main attack model for Ascon hardware

Table 3.2: Comparison of SPA, DPA, and CPA

Attack	Main idea	Strength	Limitation
SPA	Visual inspection of one or few traces	Simple and fast	Weak against noisy targets
DPA	Group traces by one-bit prediction	Good on simple leakage	Can give ghost peaks
CPA	Correlate measured and predicted leakage	Strong and reliable	Needs good leakage model

Chapter 4

Problem Statement and Objectives

The main problem studied in this BTP is whether hardware generated automatically by High-Level Synthesis leaks side-channel information differently from hardware written directly at RTL level.

The specific research questions are:

1. Can Ascon-128 be implemented on the ChipWhisperer CW305 using the reference C code and Vitis HLS?
2. Can functional hardware be produced with a unified encrypt/decrypt top function?
3. How do HLS directives such as unrolling, pipelining, and dataflow change timing, resource use, and structure?
4. How should side-channel leakage be measured later from this hardware using ChipWhisperer tools?
5. How should CPA with a Hamming Distance model be applied in the FPGA phase?
6. How should the leakage of the HLS-generated design later be compared with a pure RTL design under the same measurement conditions?

Table 4.1: Project objectives and measurable outputs

Objective	Measurable output
Build HLS Ascon hardware	Synthesizable top function and generated RTL/IP
Validate hardware preparation	Unified encrypt/decrypt design and testbench flow
Run HLS design exploration	Baseline, unrolled, pipelined, and dataflow solutions
Plan FPGA attack stage	Clear methodology for CW305 capture and CPA
Plan HLS versus RTL study	Fair comparison method for later work

Chapter 5

Platform and Design Flow

5.1 Available Boards

The local draft material records the following boards as available in the lab:

- ChipWhisperer Level 1 Starter Kit
- ChipWhisperer Level 2 Starter Kit
- ChipWhisperer CW305 Artix FPGA Target
- ChipWhisperer Nano Learning Kit

Among these, the CW305 Artix FPGA target is the correct board for the current phase because the goal is to study side-channel leakage from custom hardware, not only from firmware on a microcontroller.

Table 5.1: Board selection for this BTP

Board	Type	Suitability for current work
CW305 Artix FPGA Target	FPGA	Best choice for custom hardware and side-channel analysis
ChipWhisperer Nano	Microcontroller	Best for learning and firmware attacks, already used before
Level 1 / Level 2 Kits	Microcontroller-focused setup	Useful for software targets, not the main board for HLS hardware study

5.2 Local Project Files Used as Context

The current working folder contains the following important files that shape this report:

- `ascon_modified_codes.txt`: merged HLS-friendly Ascon code with `api.h`, `ascon_core.c`, `main.c`, and `permutations.h`
- `run_exploration_tcl.txt`: Vitis HLS automation script for design-space exploration
- `Ascon_Hardware_Test.ipynb`: prepared CW305 hardware test notebook
- `BTP LOG.docx`: draft project notes and previous context
- `Context_of_btp.txt` and `convo_of_all.txt`: planning history and design reasoning
- Earlier PDF material from the previous semester and AES CPA work

5.3 Design Flow

The design flow for the current work starts from the Ascon-128 reference C code. A unified HLS top function is prepared from the reference source. Vitis HLS is then used to generate different hardware solutions. The resulting design can be exported as RTL or packaged IP and then moved into the FPGA build flow to generate a bitstream for the CW305 platform. The later phases, namely physical board execution, trace capture from the FPGA, and CPA on measured CW305 power traces, are planned next steps and are not claimed as completed results in this report.

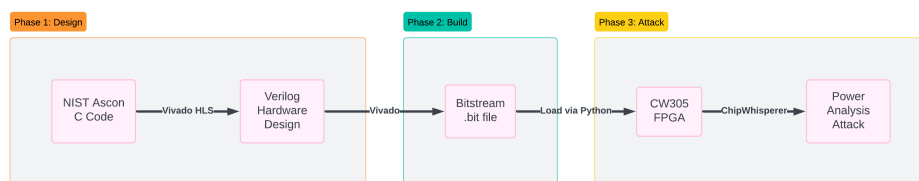


Figure 5.1: Workflow flowchart for the current project, from the Ascon reference C code to HLS generation, bitstream creation, and future FPGA-side CPA.

Table 5.2: Tool and target settings found in local files

Setting	Value
HLS project script file	<code>run_exploration_tcl.txt</code>
Top function	<code>ascon_core</code>
Target FPGA part	<code>xc7a100tftg256-2</code>
Clock constraint	10 ns
Board test notebook	<code>Ascon_Hardware_Test.ipynb</code>
Notebook hardware target	<code>cw.targets.CW305</code>
Notebook PLL setting	10 MHz

The HLS flow uses a 10 ns clock constraint, which corresponds to 100 MHz, to synthesize and analyze the hardware at a practical target speed. The physical CW305 board is planned to be run at a slower 10 MHz during measurement. This is standard practice because synthesizing the hardware fast but running the board slow helps avoid timing problems during bring-up and also gives cleaner, easier-to-capture side-channel power traces.

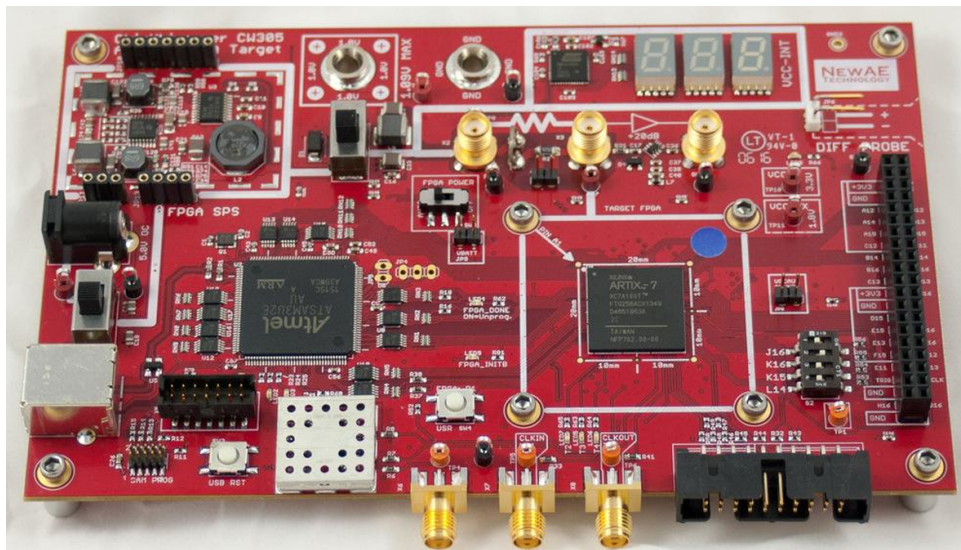


Figure 5.2: ChipWhisperer CW305 board setup prepared for FPGA programming, hardware validation, and later side-channel measurement.

Chapter 6

Ascon-128 Hardware Design

6.1 Why Ascon-128

Ascon is a lightweight authenticated encryption algorithm designed for constrained environments. It provides confidentiality and integrity in one construction. It is a suitable target for this BTP because it is modern, hardware-relevant, and smaller than many large public-key post-quantum schemes.

6.2 Reference Code Choice

The planning notes in the workspace clearly state that the selected source folder is `crypto_aead/ascon128v12/ref`. This choice is important because the reference implementation is written in clean standard C and is easier for HLS tools to synthesize than heavily optimized CPU-specific versions.

6.3 Unified Top Function

The local source file dump in `ascon_modified_codes.txt` uses a merged function:

Listing 6.1: Unified top function interface from local merged code

```
int ascon_core(unsigned char* d_out, unsigned long long*
    d_out_len,
                const unsigned char* d_in, unsigned long long
                d_in_len,
                const unsigned char* ad, unsigned long long adlen,
                const unsigned char* nsec, const unsigned char*
                npub,
                const unsigned char* k, int mode);
```

This interface is appropriate for hardware because it avoids two separate top functions and allows one hardware block to support both encryption and decryption. The `mode` input selects the required operation.

6.4 HLS Interface Pragmas

The merged code uses memory interface pragmas to guide synthesis. These pragmas are important because they tell HLS how to map input and output arrays to hardware interfaces.

Listing 6.2: HLS interface pragmas from local merged code

```
#pragma HLS INTERFACE ap_memory port=d_in depth=100
#pragma HLS INTERFACE ap_memory port=d_out depth=100
#pragma HLS INTERFACE ap_memory port=ad depth=100
#pragma HLS INTERFACE ap_memory port=npub depth=16
#pragma HLS INTERFACE ap_memory port=k depth=16
```

6.5 Operational Stages

The merged design follows the standard Ascon flow:

1. Length calculation according to mode
2. Shared initialization
3. Shared associated-data processing
4. Data processing with encrypt/decrypt branch
5. Shared finalization
6. Tag output or tag verification

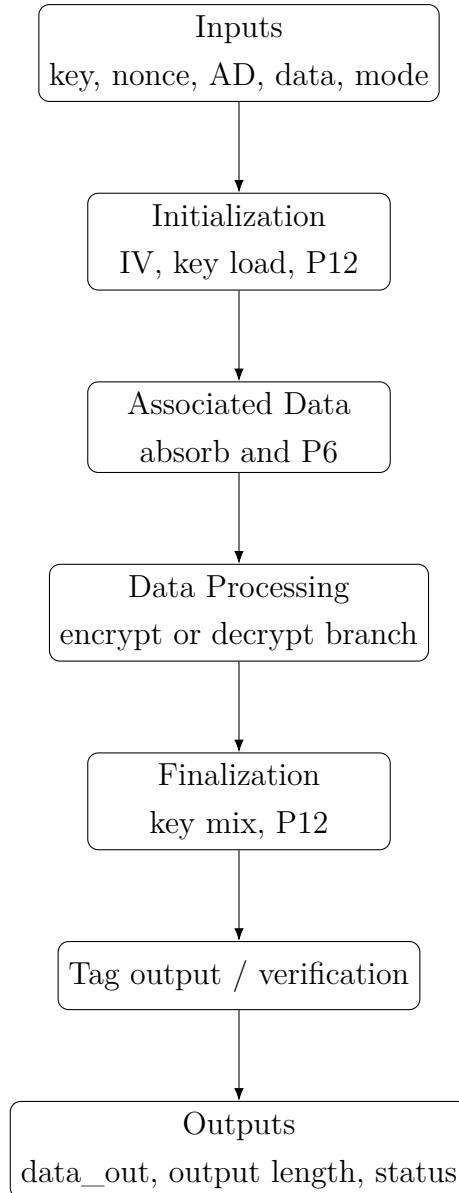
Figure 6.1: Stage-level structure of the merged `ascon_core`

Table 6.1: Local source components and their roles

File/component	Role
<code>api.h</code>	Defines sizes and declares the unified hardware top function
<code>ascon_core.c</code>	Main merged encrypt/decrypt logic for HLS synthesis
<code>main.c</code>	Functional testbench for software/HLS simulation style checking
<code>permutations.h</code>	State definition, byte-word conversion, permutation support functions

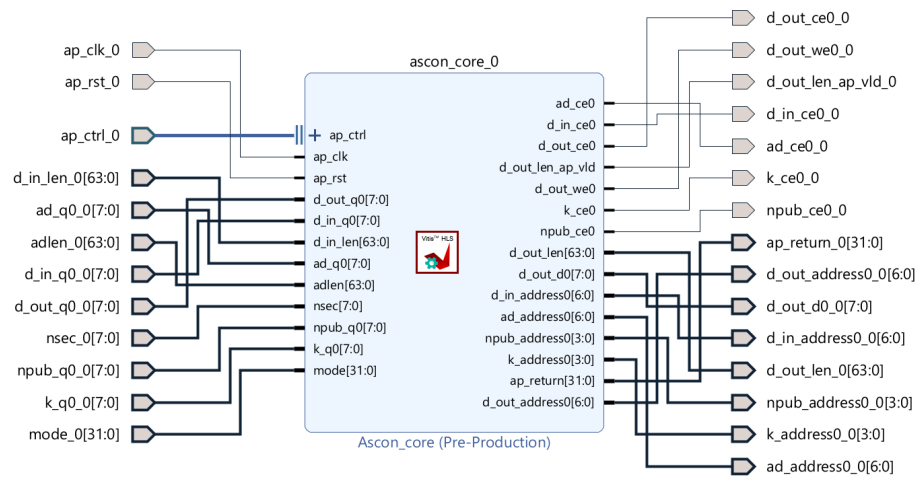


Figure 6.2: IP core interface diagram of the generated `ascon_core` hardware block showing the main ports used for system integration.

Chapter 7

Side-Channel Attack Methodology

7.1 Attack Choice

The selected future attack is Correlation Power Analysis. This choice is appropriate because the target is a clocked hardware design and the project aims to measure not only whether leakage exists, but also how strongly measured power matches a transition-based leakage model.

7.2 Leakage Model

The main leakage model planned for the FPGA phase is Hamming Distance. For a chosen internal transition, the attacker computes:

$$\text{HD}(x_{old}, x_{new}) = \text{HW}(x_{old} \oplus x_{new}) \quad (7.1)$$

This captures how many bits switch between two states. In FPGA-based cryptographic hardware, such transitions often align well with dynamic power consumption.

7.3 Target Intermediate Selection

The exact best intermediate depends on final implementation details, but the likely targets include:

- a state word before and after a permutation round,
- the output of a nonlinear substitution layer,
- a register update around plaintext or ciphertext absorption,

- a data-dependent transition inside the round function.

7.4 General CPA Flow

The planned CPA flow for the FPGA phase is straightforward in concept, even though the physical measurements are not yet completed. Known inputs such as plaintext, nonce, associated data, and operation mode will be chosen first. Power traces will then be captured from the CW305 while the Ascon core runs. For each candidate guess or chosen internal transition, a Hamming Distance leakage vector will be computed. Pearson Correlation will then be evaluated across time samples, and the strongest peak will be used to identify the most likely hypothesis.

Listing 7.1: Pseudo-code for CPA with Hamming Distance

```
for guess in candidate_space:
    hyp = []
    for trace_input in all_inputs:
        old_state, new_state = model_internal_transition(
            trace_input, guess)
        hyp.append(hamming_weight(old_state ^ new_state))
    for t in range(num_samples):
        corr[guess][t] = pearson(hyp, traces[:, t])
```

Table 7.1: Planned attack inputs and outputs for the FPGA phase

Item	Description
Known inputs	Plaintext or ciphertext, nonce, associated data, and mode control
Secret input	Key
Measured data	Future CW305 power traces
Predicted data	Hamming Distance of selected internal transitions
Main result	Correlation peak and candidate ranking after physical measurement

Chapter 8

Experimental Setup

8.1 HLS Design-Space Exploration

The local file `run_exploration_tcl.txt` contains a Vitis HLS script that builds four solutions for the same top function. These solutions are:

1. Baseline
2. Unrolled
3. Pipelined
4. Dataflow

Listing 8.1: Local HLS exploration script excerpt

```
open_project ascon_hardware_dse
set_top ascon_core
add_files ascon_core.c

open_solution "solution1_baseline"
set_part {xc7a100tftg256-2}
create_clock -period 10 -name default
csynth_design

open_solution "solution2_unrolled"
set_part {xc7a100tftg256-2}
create_clock -period 10 -name default
set_directive_unroll -factor 2 "ascon_core/data_loop"
csynth_design
```

Table 8.1: HLS solutions found in the local script

Solution	Directive	Expected impact
solution1_baseline	none	Reference design for timing and area
solution2_unrolled	loop unroll factor 2	More parallel work, possible area increase
solution3_pipelined	loop pipeline	Better throughput, control changes
solution4_dataflow	function-level dataflow	More overlap between stages

8.2 Planned CW305 Hardware Test Flow

The local notebook `Ascon_Hardware_Test.ipynb` shows the intended board test path for the next phase. It connects to the board over USB, flashes a bitstream, configures voltage and PLL, writes the key and plaintext into hardware registers, starts the hardware, and reads back ciphertext. This notebook is part of the prepared methodology in the current workspace, but it is not presented here as completed physical measurement evidence.

Listing 8.2: Notebook commands prepared for CW305 hardware test

```
target = cw.target(None, cw.targets.CW305)
target.fpga.FPGAProgram(open("cw305_top.bit", "rb").read())
target.vccint_set(1.0)
target.pll.pll_enable_set(True)
target.pll.pll_outenable_set(True, 1)
target.pll.pll_outfreq_set(10E6, 1)
target.fpga_write(target.REG_CRYPT_KEY, secret_key)
target.fpga_write(target.REG_CRYPT_TEXTIN, plaintext)
target.go()
ciphertext = target.fpga_read(target.REG_CRYPT_CIPHEROUT, 16)
```

Table 8.2: Local hardware test settings from the prepared notebook

Parameter	Value
Target board	CW305
Bitstream name in notebook	<code>cw305_top.bit</code>
Core voltage	1.0 V
PLL output frequency	10 MHz
Test key length	16 bytes
Test plaintext length shown	16 bytes in notebook example

8.3 Planned Capture Conditions

The final capture script for CPA is not present in the current folder, so the measurement section is recorded here only as the intended setup for future work:

- fixed key across one attack experiment,
- chosen or random plaintext set,
- stable trigger position,
- same clock condition across all compared implementations,
- same gain, sample window, and number of traces when comparing HLS and RTL.

Chapter 9

Results and Analysis

At the time of preparing this report, the current workspace contains completed HLS exploration data and prepared hardware-test methodology, but it does not contain completed physical CW305 power-trace results, FPGA CPA plots, or pure RTL comparison measurements. Therefore, this chapter reports only what is already available and states the remaining attack stages as future work.

9.1 Functional Validation

The merged code includes a software-style testbench in `main.c`. It performs encryption, then decryption, and checks whether the original plaintext is recovered. This is the first validation step before hardware-level testing and trace capture. The available local code shows that the design effort has already reached the stage of unified functional behavior and synthesis preparation, even though the final physical board execution results are still pending.

9.2 Synthesis Results

Table 9.1: HLS synthesis results obtained from the completed design exploration stage

Solution	LUTs	FFs	Fmax	Estimated latency
Baseline	8,075	3,923	123.7 MHz	~20 cycles
Unrolled	11,540	5,160	123.5 MHz	~10 cycles
Pipelined	11,811	7,823	137.2 MHz	~22 cycles
Dataflow	10,937	4,816	125.2 MHz	~18 cycles

These HLS results show that the design-space exploration stage is complete and already gives useful engineering insight. Loop unrolling reduces estimated cycle count but increases hardware cost. Pipelining improves the maximum reported frequency and also changes the control structure significantly. Dataflow also changes internal concurrency while keeping area lower than the pipelined case. These differences are important not only for performance but also because they may influence switching activity and, later, side-channel leakage behavior.

9.3 Future Hardware Measurement Stage

The next stage after bitstream generation is physical execution on the CW305, followed by trace capture and CPA on the FPGA implementation. This stage is not yet complete. When it is performed, the important factors will be trigger stability, clock consistency, the selection of the internal transition used in the Hamming Distance model, and the number of traces required to obtain a stable correlation result. At present, these steps are part of the research methodology and future execution plan, not reported results.

Chapter 10

Comparison of HLS Hardware and RTL Hardware

The main long-term goal of the project is to compare HLS-generated hardware against a pure RTL implementation. This comparison has not yet been executed because the physical FPGA leakage stage and the pure RTL implementation stage are both pending. Therefore, this chapter records only the comparison method and its research importance.

10.1 Fair Comparison Rules

1. Use the same Ascon-128 algorithm variant.
2. Use the same board family and as similar a wrapper as possible.
3. Use the same clock or a clearly reported clock condition.
4. Use the same key/plaintext experiment plan.
5. Use the same capture hardware settings.
6. Use the same correlation method and leakage model.

At present, no RTL source files, RTL implementation reports, or RTL trace-capture results are available in the current workspace. The comparison will become meaningful only after both implementations are tested under the same clock, same board conditions, same input plan, and same CPA script. The final comparison will study whether the automatic scheduling and control structure introduced by HLS affects visible side-channel leakage differently from a handwritten RTL design.

Chapter 11

Challenges, Limits, and Validity

This work has several practical limits that must be stated clearly.

Table 11.1: Threats to validity and mitigation

Issue	Impact	Mitigation
Trace noise	Weak correlation peaks	Better grounding, more traces, stable clock
Alignment errors	Smeared leakage	Trigger tuning and post-alignment
Model mismatch	Wrong attack result	Test multiple HD targets and internal transitions
HLS structural changes	Hard to isolate leakage cause	Compare across multiple directives and later with RTL
Missing RTL data in current workspace	Incomplete comparison	Add RTL build and result data later

Chapter 12

Conclusion and Future Work

This BTP moves from firmware side-channel analysis on the ChipWhisperer Nano to hardware design for side-channel analysis on the ChipWhisperer CW305 FPGA target. The selected cryptographic target is Ascon-128, and the selected future attack method is Correlation Power Analysis with the Hamming Distance leakage model. The project is built around a practical hardware design path that starts from the reference C implementation and uses Vitis HLS to generate hardware. The local workspace already contains a merged top-level Ascon hardware function, a Tcl design exploration script, and a prepared CW305 test notebook, which together form a solid base for the next stage.

The completed part of the current phase is the design and synthesis side. The HLS exploration already shows meaningful trade-offs between baseline, unrolled, pipelined, and dataflow implementations. The next phase is physical board execution, trace capture, and CPA on the FPGA implementation. After that, a pure RTL version should be prepared so that a fair HLS versus RTL leakage comparison can be performed under matched conditions.

Future work includes final RTL implementation integration, large-scale trace capture, attack-point refinement inside the Ascon permutation flow, and the study of hardware countermeasures such as masking, balancing, or randomization.

References

- [R1] Ascon reference implementation and specification documents.
- [R2] NIST Lightweight Cryptography project material for Ascon.
- [R3] ChipWhisperer documentation for CW305 and capture workflow.
- [R4] Xilinx Vivado and Vitis HLS documentation.
- [R5] Standard texts and papers on power analysis, DPA, CPA, and leakage models.